

packages/graph-view/src/utils/layout.ts · Code explanation (L231-266)

Kind: modified · Baseline: 8e0dc031dcfc · Target: NovaDiff · Requested mode: exact · Effective mode: exact ·
Generated: 2026-05-19T09:52:26.866Z

What it does `mergeElkPositions` takes an array of graph nodes (`nodes`) and an `ElkInput` result (`positioned`) that contains layout coordinates from the ELK layout engine. It returns a new array of nodes where each node's `position` (and optionally `width / height` for containers) is replaced with the coordinates supplied by ELK. If ELK did not provide a position for a node, the original node is returned unchanged, defaulting its `position` to `{x:0,y:0}` if missing.

How it works1. Build a lookup map

Each child from `positioned` is stored by its `id` with its coordinates and optional size.2. **Map over the original nodes**For each node, it looks up the ELK result. If found, it replaces the node's `position` with the ELK coordinates and, for container nodes, copies back the `width / height` that ELK echoed. If not found, the node is left untouched except for ensuring a default position.

Notes for reviewers- The function mutates no input objects; it returns a new array with new node objects.- Width/height propagation is only applied when ELK supplies them (`merged.width !== null`). This is intentional for container nodes to allow a subsequent tick-driven re-layout to resize the visible atom.- The default `{x:0,y:0}` fallback ensures nodes without ELK data still have a defined position, preventing rendering glitches.- The use of `positioned.children ?? []` guards against `children` being `undefined` .- No side effects beyond the returned array; the function is pure.